E-Commerce Customer Behavior and Recommendation Analytics

Project Report

Subject: Big Data

Submitted by

Ronit (23CSU1171)
Krish Gaur (23CSU162)

Github Repo :- https://github.com/ronitx05/ecommerce-customer-analytics

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING,

SCHOOL OF ENGINEERING AND TECHNOLOGY

THE NORTHCAP UNIVERSITY

GURUGRAM-122017

INDEX

1. Abstract 4
2. Introduction 4
2.1 Background 4
2.2 Problem Statement 4
2.3 Objectives 4
2.4 Target Users and Expected Outcomes 5
3. Dataset Description 5
3.1 Source and Citation 5
3.2 Attributes 6
3.3 Suitability for Big Data Processing 6
4. Tools and Technologies 6
5. System Architecture and Methodology 7
6. Data Ingestion: Bronze Layer 7
7. Data Preprocessing: Silver Layer 8
7.1 Data Quality Issues Found 8
7.2 Cleaning Steps 8
7.3 Result 8
8. PySpark Processing and Gold Layer 9
9. Exploratory Data Analysis 10
9.1 Sales Trend 10
9.2 Top Products 10
9.3 Country Analysis 11
9.4 Ordering Patterns 11
9.5 RFM Distributions 11
10. Machine Learning with Spark MLlib 12
10.1 Customer Segmentation using K-Means 12
10.2 Product Recommendation using ALS 13
11. Interactive Dashboard 14
11.1 Dashboard Screenshots 14
12. Automation and Cloud Deployment 17

12.1 Pipeline Automation 17
12.2 Apache Airflow 17
12.3 Google Cloud Dataproc 17
13. Version Control 18
14. Results and Business Insights 18
15. Conclusion and Future Scope 18
16. References 19

1. Abstract

Online retailers record millions of transactions, but turning this raw data into decisions about customers is difficult without scalable processing. This project builds an end-to-end Big Data pipeline on the **UCI Online Retail II** dataset (1,067,371 transactions from a UK-based online retailer, December 2009 to December 2011) using **Apache Spark (PySpark)**. Data flows through a **Bronze, Silver and Gold (medallion) architecture**: raw ingestion with lineage metadata, cleaning and validation, and business-ready aggregates built with **Spark SQL and window functions**.

On top of the Gold layer, **Spark MLlib** is used for two tasks: **K-Means clustering** on Recency, Frequency and Monetary (RFM) features to segment 5,852 customers into four actionable groups (silhouette score 0.529), and **ALS collaborative filtering** to recommend five unseen products to every customer (hit-rate@10 of 0.499 on a 20% hold-out set). Results are delivered through an interactive **Streamlit dashboard** and a Jupyter notebook, the pipeline is automated with shell/PowerShell scripts and an **Apache Airflow** DAG, and a **Google Cloud Dataproc** submission script is provided for cloud execution.

2. Introduction

2.1 Background

E-commerce companies compete on customer experience. Knowing which customers are most valuable, which are about to leave, and which products each customer is likely to buy next directly affects marketing spend, retention and revenue. Transaction logs contain this information, but at the scale of millions of rows, traditional single-machine tools become slow and difficult to scale. Distributed frameworks such as Apache Spark make it possible to clean, aggregate and model such data efficiently and to move the same code from a laptop to a cloud cluster.

2.2 Problem Statement

Given about one million raw e-commerce transactions containing noise (missing customer IDs, cancellations, invalid quantities and duplicates), design a scalable Big Data pipeline that (a) cleans and organises the data, (b) identifies sales trends and customer purchasing behaviour, (c) segments customers by value and engagement, and (d) generates personalised product recommendations, and presents the results to business users through an interactive dashboard.

2.3 Objectives

- Ingest the raw dataset into a Bronze layer while preserving source and ingestion metadata.

- Clean and validate the data into a Silver layer (types, duplicates, missing and invalid values, cancellations).
- Use PySpark and Spark SQL (joins, aggregations, window functions) to build Gold-layer business tables.
- Compute RFM features and segment customers with K-Means (Spark MLlib).
- Build an ALS recommendation model, validate it on held-out data and store predictions in Parquet.
- Present KPIs, trends, segments and recommendations in a Streamlit dashboard.
- Automate the pipeline and maintain the project on GitHub; provide Airflow and Google Cloud Dataproc deployment.

2.4 Target Users and Expected Outcomes

Target users: marketing and CRM teams (customer segments, win-back campaigns), merchandising teams (product and country performance) and management (KPIs and trends).

Expected outcomes: a clean, analysis-ready dataset; KPI and trend reports; four interpretable customer segments; top-5 personalised product recommendations for every customer; and a dashboard that uses the actual pipeline outputs.

3. Dataset Description

3.1 Source and Citation

The project uses the **Online Retail II** dataset from the UCI Machine Learning Repository. It contains all transactions of a UK-based, non-store online retailer that mainly sells unique all-occasion giftware; many of its customers are wholesalers.

Property	Value
Name	Online Retail II
Source	UCI Machine Learning Repository (archive.ics.uci.edu/dataset/502)
Citation	Chen, D. (2012). Online Retail II. https://doi.org/10.24432/C5CG6D
Licence	Creative Commons Attribution 4.0 (CC BY 4.0)
Time period	01/12/2009 to 09/12/2011
Size	1,067,371 rows, 8 attributes, 2 Excel sheets (about 45 MB compressed)

Table 3.1: Dataset summary

3.2 Attributes

Attribute	Type	Description
Invoice	Nominal	6-digit invoice number; a leading 'C' marks a cancellation
StockCode	Nominal	5-digit product code (plus optional letter)
Description	Nominal	Product name
Quantity	Numeric	Units of the product in the transaction line
InvoiceDate	Date-time	Date and time of the invoice
Price	Numeric	Unit price in pounds sterling (GBP)
Customer ID	Nominal	5-digit customer identifier
Country	Nominal	Country where the customer resides

Table 3.2: Dataset attributes

3.3 Suitability for Big Data Processing

With over one million transaction lines, 5,900+ customers, 4,600+ products and 41 countries, the dataset is large enough to demonstrate distributed processing: shuffles for aggregations and joins, window functions over the full dataset, partitioned storage, and iterative MLlib algorithms (ALS on about 480,000 customer-product interactions). The same code runs unchanged on a multi-node Dataproc cluster.

4. Tools and Technologies

Technology	Purpose in the project
Apache Spark 4 / PySpark	Distributed processing engine for all pipeline stages
Spark SQL	Declarative analytics, joins, aggregations and window functions (LAG, SUM OVER, DENSE_RANK, NTILE, ROW_NUMBER)
Spark MLlib	VectorAssembler, StandardScaler, K-Means, ALS, ClusteringEvaluator, RankingEvaluator, RegressionEvaluator
Apache Parquet	Columnar storage for Bronze, Silver and Gold layers (Silver partitioned by month)
Pandas / openpyxl	One-time conversion of the source Excel sheets to CSV
Streamlit + Plotly	Interactive dashboard over the Gold and prediction outputs
Jupyter Notebook + Matplotlib	Step-by-step exploratory analysis and model walkthrough
Apache Airflow	DAG for scheduled orchestration of the four pipeline stages
Google Cloud Dataproc + Cloud Storage	Managed Spark cluster and data lake for cloud execution
Git + GitHub	Version control using feature branches and pull requests

Table 4.1: Technology stack

5. System Architecture and Methodology

The project follows the **medallion architecture**. Each layer is stored as Parquet and is produced by a separate PySpark script, so every stage can be re-run, scheduled or moved to the cloud independently.

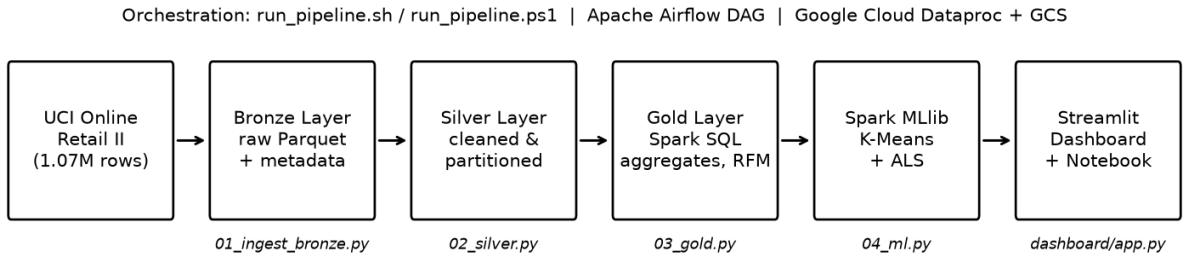


Figure 5.1: End-to-end pipeline architecture

Step	Task	Implementation
1	Domain and problem	E-commerce customer behaviour; problem statement in Section 2
2	Collect and ingest	Automatic download from UCI; Excel to CSV landing (01_ingest_bronze.py)
3	Bronze layer	Raw Parquet with source_system, source_file, ingested_at
4	Silver layer	Type casting, de-duplication, invalid/missing records removed (02_silver.py)
5	PySpark processing	Filters, joins, aggregations, Spark SQL, window functions
6	Gold layer	KPIs, monthly trends, product/country stats, customer RFM (03_gold.py)
7	Analytics / MLlib	K-Means segmentation and ALS recommendations (04_ml.py)
8	Predictions and validation	Silhouette, precision@10, MAP@10, hit-rate@10; outputs in Parquet
9	Dashboard	Streamlit app with five tabs (dashboard/app.py)
10	Integrate and automate	run_pipeline.sh / .ps1, Airflow DAG, Dataproc script, GitHub

Table 5.1: Mapping of project steps to implementation

6. Data Ingestion: Bronze Layer

The ingestion script downloads the source zip file from UCI if it is not already present, reads both Excel sheets (Year 2009-2010 and Year 2010-2011) and lands them as a single CSV file. Spark then reads the CSV, normalises the column names (for example 'Customer ID' becomes customer_id) and writes the data as Parquet **without changing any values**: all columns are kept as strings so that nothing is lost before validation.

Three lineage columns are added to every record: **source_system** (dataset name and DOI), **source_file** and **ingested_at** (ingestion timestamp). This makes every downstream record traceable to its origin. The Bronze layer contains **1,067,371 rows**.

```
df = spark.read.option("header", True).csv(CSV_PATH)
```

```
df = (df.withColumn("source_system", F.lit(SOURCE_NAME))
        .withColumn("source_file", F.lit("online_retail_ii.csv"))
        .withColumn("ingested_at", F.current_timestamp()))
df.write.mode("overwrite").parquet(BRONZE_DIR)
```

7. Data Preprocessing: Silver Layer

7.1 Data Quality Issues Found

Issue in raw data	Rows affected
Missing Customer ID	243,007
Missing Description	4,382
Exact duplicate rows	34,335
Cancellation lines (Invoice starting with 'C')	19,494
Quantity less than or equal to zero	22,950
Price less than or equal to zero	6,207

Table 7.1: Data quality issues in the Bronze layer

7.2 Cleaning Steps

- **Type casting:** quantity to integer, price to double, invoice date to timestamp, customer ID to integer.
- **Standardisation:** trimmed and upper-cased stock codes and descriptions; country names in title case.
- **De-duplication:** exact duplicate rows removed with dropDuplicates().
- **Invalid records:** rows without a customer ID or valid timestamp removed, because they cannot be used for customer analytics.
- **Non-product codes:** postage, carriage, bank charges, Amazon fees, manual adjustments and test codes removed.
- **Cancellations:** separated into their own Silver table and used later to compute cancellation rates.
- **Derived columns:** line_total = quantity x price, invoice_date and year_month; sales data partitioned by year_month.

7.3 Result

Stage	Rows
Bronze (raw)	1,067,371
After removing exact duplicates	1,033,036
Silver sales (valid customer, quantity > 0, price > 0, real products)	776,579
Silver cancellations (stored separately)	17,602

Table 7.2: Row counts after cleaning

A data quality report (quality_report.json) is written with these counts and displayed on the dashboard.

8. PySpark Processing and Gold Layer

The Gold layer is built entirely with **Spark SQL** on temporary views of the Silver tables. It demonstrates aggregations, a **join** between sales and cancellations, and several **window functions**.

Gold table	Content	Spark techniques
kpis	Revenue, orders, customers, products, countries, average order value	Aggregations, COUNT DISTINCT
monthly_revenue	Revenue, orders and customers per month with growth and running total	LAG() OVER, SUM() OVER
product_stats	Units, revenue and buyers per product with revenue rank	GROUP BY, DENSE_RANK()
country_stats	Revenue, orders, customers and cancellation rate per country	LEFT JOIN of sales and cancellations
top_products_by_country	Top 5 products in every country	DENSE_RANK() PARTITION BY country
time_patterns	Orders and revenue by weekday and hour	Date functions, aggregation
customer_rfm	Recency, frequency, monetary, 1-5 scores and rule-based segment	DATEDIFF, NTILE(5), CASE

Table 8.1: Gold layer tables

Example: month-over-month revenue growth using the LAG window function.

```
SELECT year_month, revenue,
       ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY year_month))
             / LAG(revenue) OVER (ORDER BY year_month), 2) AS mom_growth_pct,
       SUM(revenue) OVER (ORDER BY year_month) AS cumulative_revenue
FROM monthly
```

Overall KPIs from the Gold layer: **total revenue GBP 17.07 million**, **36,594 orders**, **5,852 customers**, **4,620 products**, **41 countries** and an **average order value of GBP 466.43**.

9. Exploratory Data Analysis

9.1 Sales Trend

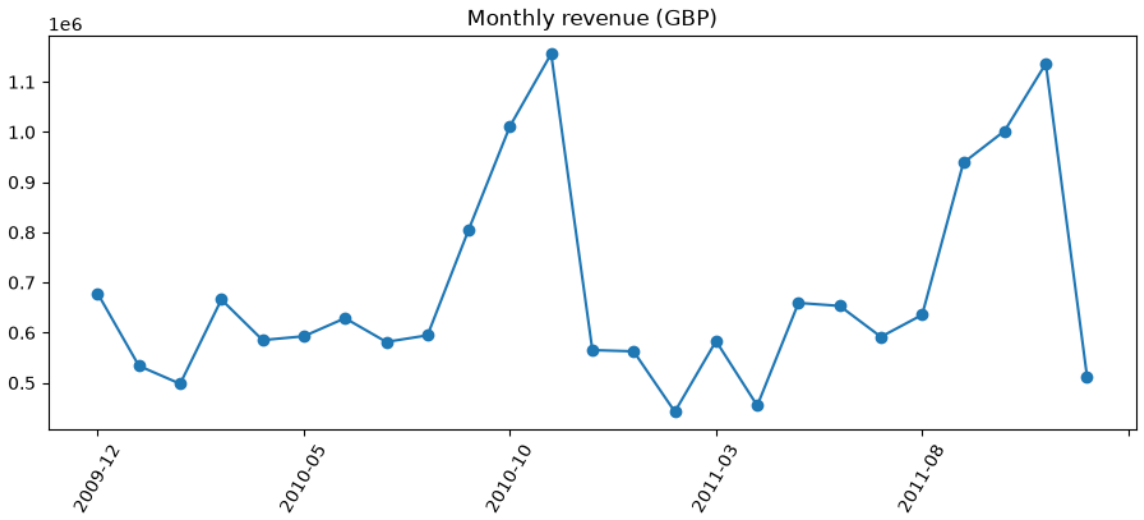


Figure 9.1: Monthly revenue (GBP)

Revenue shows strong **seasonality**. The two highest months are November 2010 (GBP 1.16M) and November 2011 (GBP 1.14M), followed by October of both years, which reflects pre-Christmas gift buying. The low value for December 2011 is only because the data ends on 9 December 2011.

9.2 Top Products

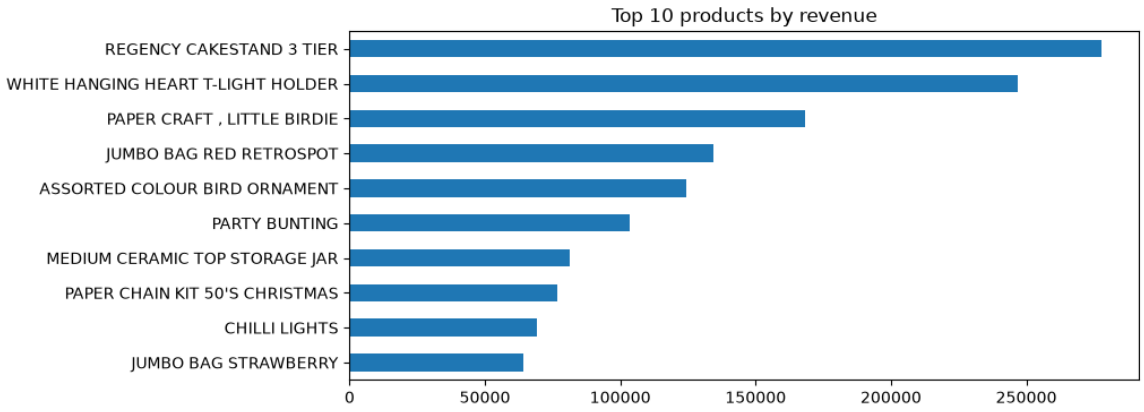


Figure 9.2: Top 10 products by revenue

The best-selling product by revenue is the **Regency Cakestand 3 Tier** (GBP 277,656), followed by the **White Hanging Heart T-Light Holder** (GBP 247,203). High-volume, low-price items such as the Jumbo Bag Red Retrospot sell more than 93,000 units.

9.3 Country Analysis

Country	Revenue (GBP)	Share	Customers	Cancel rate
United Kingdom	14,288,774	83.7%	5,334	16.3%
EIRE	586,626	3.4%	3	20.1%
Netherlands	549,773	3.2%	22	4.9%
Germany	383,289	2.2%	107	27.3%
France	309,403	1.8%	93	15.8%

Table 9.1: Top 5 countries by revenue

The United Kingdom generates about 84% of revenue. EIRE and the Netherlands have very few but very large (wholesale) customers. Germany has the highest cancellation rate among the major markets.

9.4 Ordering Patterns

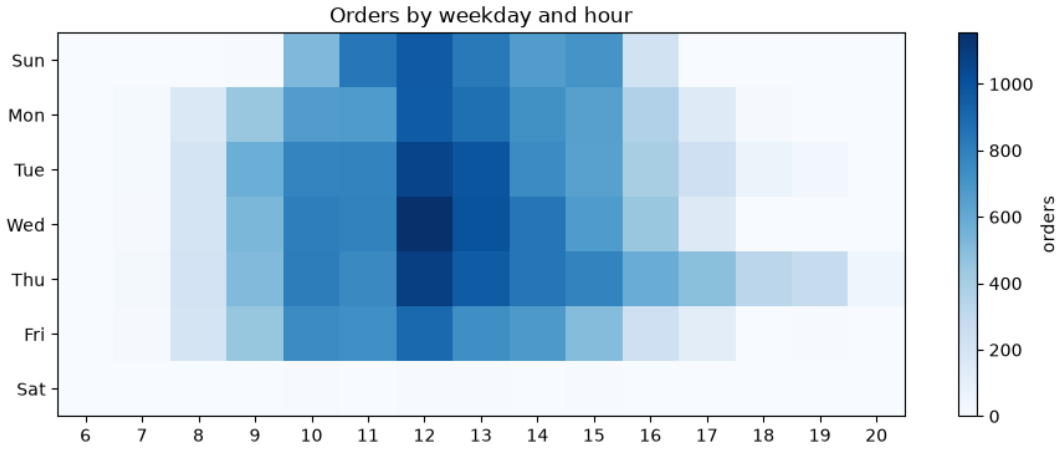


Figure 9.3: Number of orders by weekday and hour

Orders are placed almost entirely during UK business hours, peaking around midday, and there are almost no orders on Saturdays. This is consistent with a customer base made up largely of business buyers.

9.5 RFM Distributions

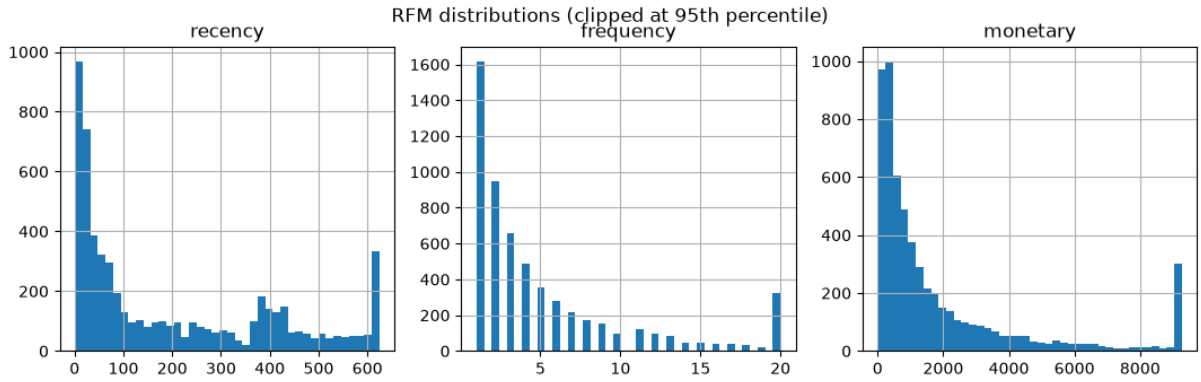


Figure 9.4: Distributions of recency, frequency and monetary value

All three RFM features are heavily right-skewed: most customers buy rarely and spend little, while a small group buys very often. A log transformation followed by standard scaling was therefore applied before clustering.

10. Machine Learning with Spark MLlib

10.1 Customer Segmentation using K-Means

Features: log(1 + recency), log(1 + frequency) and log(1 + monetary) for each customer. **Pipeline:** VectorAssembler, StandardScaler (mean 0, standard deviation 1) and KMeans (seed 42). The number of clusters k was selected using the **silhouette score** computed by ClusteringEvaluator.

k	2	3	4	5	6	7	8
Silhouette	0.626	0.505	0.529	0.510	0.501	0.480	0.467

Table 10.1: Silhouette score for different values of k

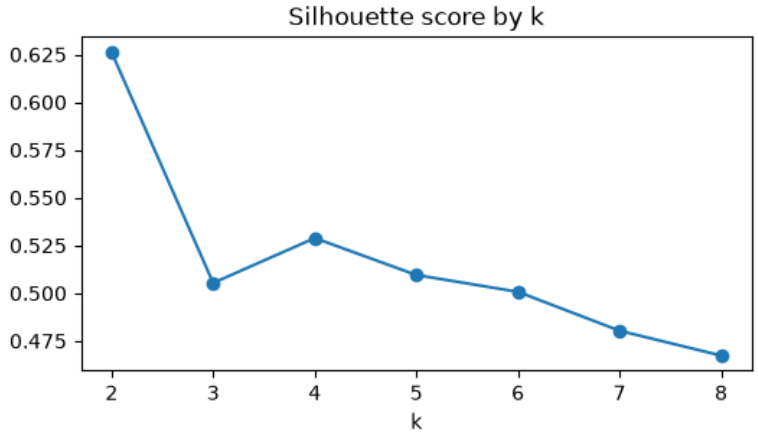


Figure 10.1: Silhouette score by number of clusters

k = 2 has the highest score but only separates active from inactive customers. **k = 4** gives the best score among k of 3 or more and produces segments that are more useful for marketing, so it was chosen.

Segment	Customers	Avg recency (days)	Avg orders	Avg spend (GBP)
High-Value Loyal	1,165	29	19.4	10,727
Mid-Value Slipping	1,430	238	5.0	1,945
Recent Low-Spend	1,283	28	3.3	919
Lapsed / Lost	1,974	388	1.4	310

Table 10.2: K-Means cluster profiles (k = 4)

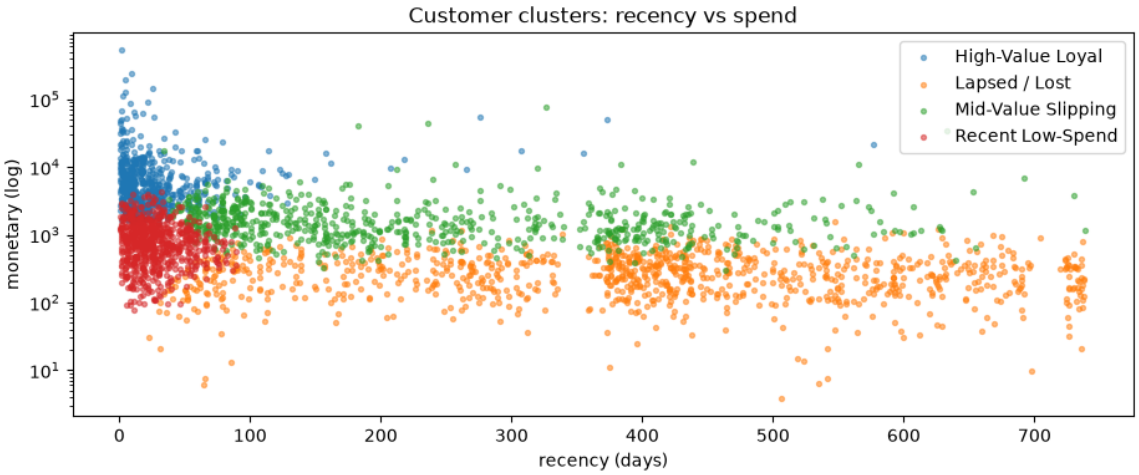


Figure 10.2: Customer clusters, recency against spend (sample of 3,000 customers)

10.2 Product Recommendation using ALS

Customers do not give explicit ratings, so **implicit feedback** was used: the rating for a customer-product pair is the number of separate orders in which the customer bought that product. Stock codes were converted to integer IDs with StringIndexer. The ALS model was configured with implicitPrefs = true, rank = 20, maxIter = 10, regParam = 0.1, alpha = 10 and coldStartStrategy = drop.

Validation: the interactions were randomly split 80/20. The model was trained on 80% and, for every test customer, the top-10 recommended products were compared with the products actually bought in the 20% hold-out set.

Metric	Value	Meaning
Customers / products / interactions	5,852 / 4,620 / 480,973	Size of the interaction matrix
Hit-rate@10	0.499	Share of customers with at least one correct product in their top 10
Precision@10	0.076	Average share of the top 10 that were actually bought
MAP@10	0.046	Mean average precision, rewarding correct items ranked higher
RMSE (implicit)	1.959	Reported for completeness; ranking metrics are more relevant for implicit ALS

Table 10.3: ALS validation metrics

A hit-rate of about 50% means that, for half of all customers, at least one product they really bought appeared in a top-10 list chosen from 4,620 products. The final model is retrained on all data and the **top 5 products the customer has not bought before** are stored for every customer.

Rank	Recommended product	Score
1	DOORMAT ENGLISH ROSE	0.872
2	DOORMAT TOPIARY	0.856
3	DOORMAT HEARTS	0.846
4	DOOR MAT MULTICOLOUR STRIPE	0.793
5	DOORMAT KEEP CALM AND COME IN	0.723

Table 10.4: Sample recommendations for customer 12346

The recommendations for this customer come from the same product family (doormats), which shows that the model has learned meaningful similarities between products from purchase behaviour alone.

All model outputs (customer_segments, cluster_profile, recommendations) are stored as Parquet files in the gold/predictions folder, and the metrics are saved in metrics.json.

11. Interactive Dashboard

A **Streamlit** dashboard (dashboard/app.py) reads the actual Gold and prediction Parquet files produced by the pipeline. It shows five KPI cards (revenue, orders, customers, products and average order value) and five tabs:

Tab	Contents
Sales Trends	Monthly revenue, month-over-month growth, active customers per month, weekday by hour heatmap
Products and Countries	Top-N products (slider), top countries, cancellation rates, top products per selected country
Customer Segments	K-Means cluster profiles, customers per cluster, recency against spend scatter plot, RFM segment against cluster
Recommendations	Select a customer to see their cluster, RFM values, purchase history and top-5 ALS recommendations
Data Quality and Model Metrics	Silver quality report and model validation metrics

Table 11.1: Dashboard tabs

The dashboard is started with the command: streamlit run dashboard/app.py.

11.1 Dashboard Screenshots

The screenshots below were taken from the running dashboard (http://localhost:8501) and show each part of the application.

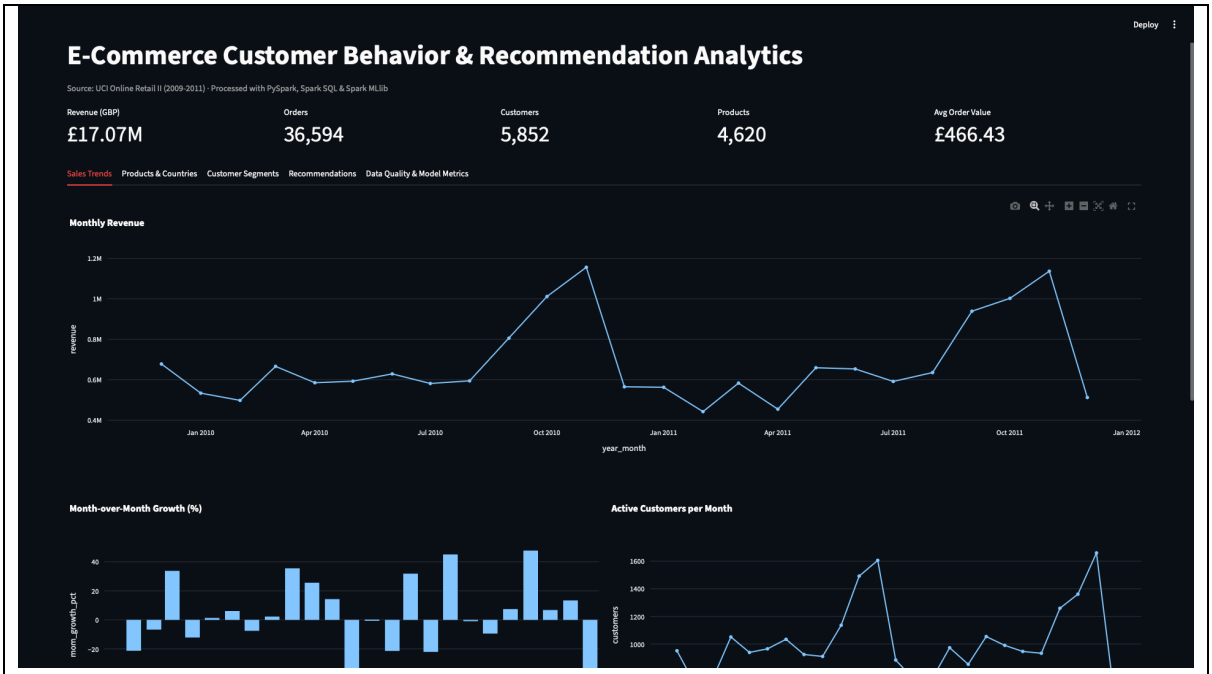


Figure 11.1: Dashboard, KPI cards and Sales Trends tab (Monthly revenue, growth and the weekday by hour heatmap)

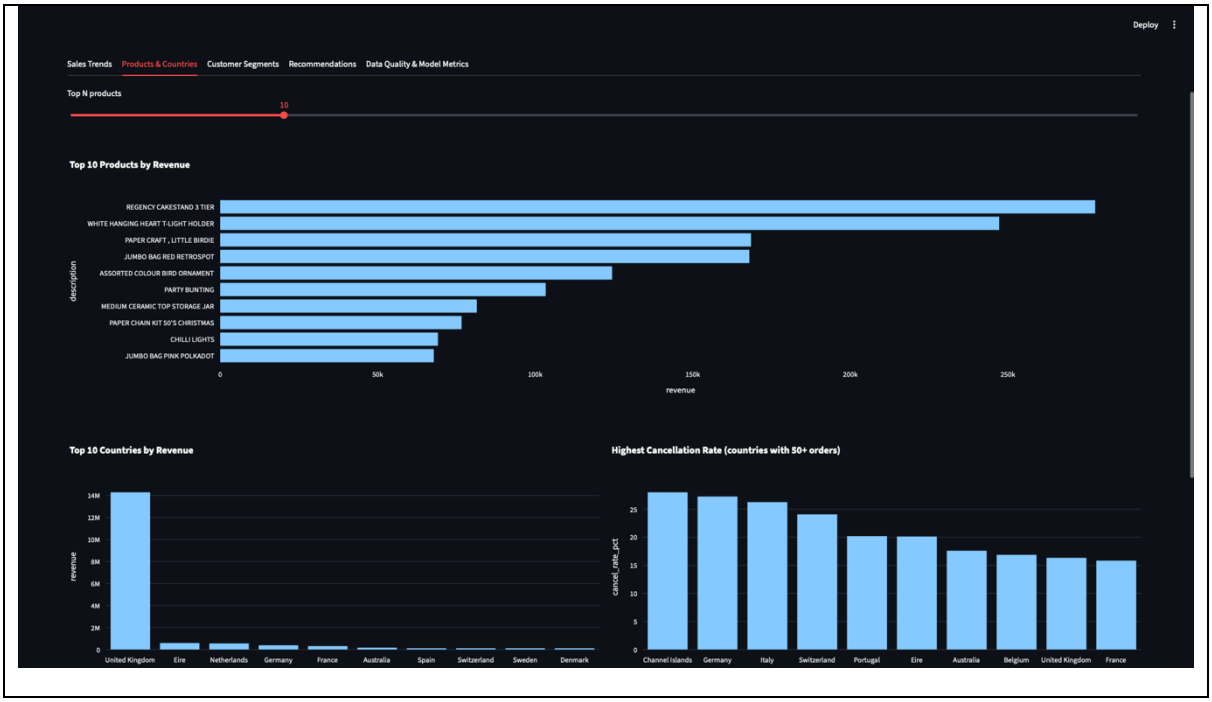


Figure 11.2: Dashboard, Products and Countries tab (Top products, top countries and cancellation rates)



Figure 11.3: Dashboard, Customer Segments tab (K-Means cluster profiles and the recency against spend scatter plot)

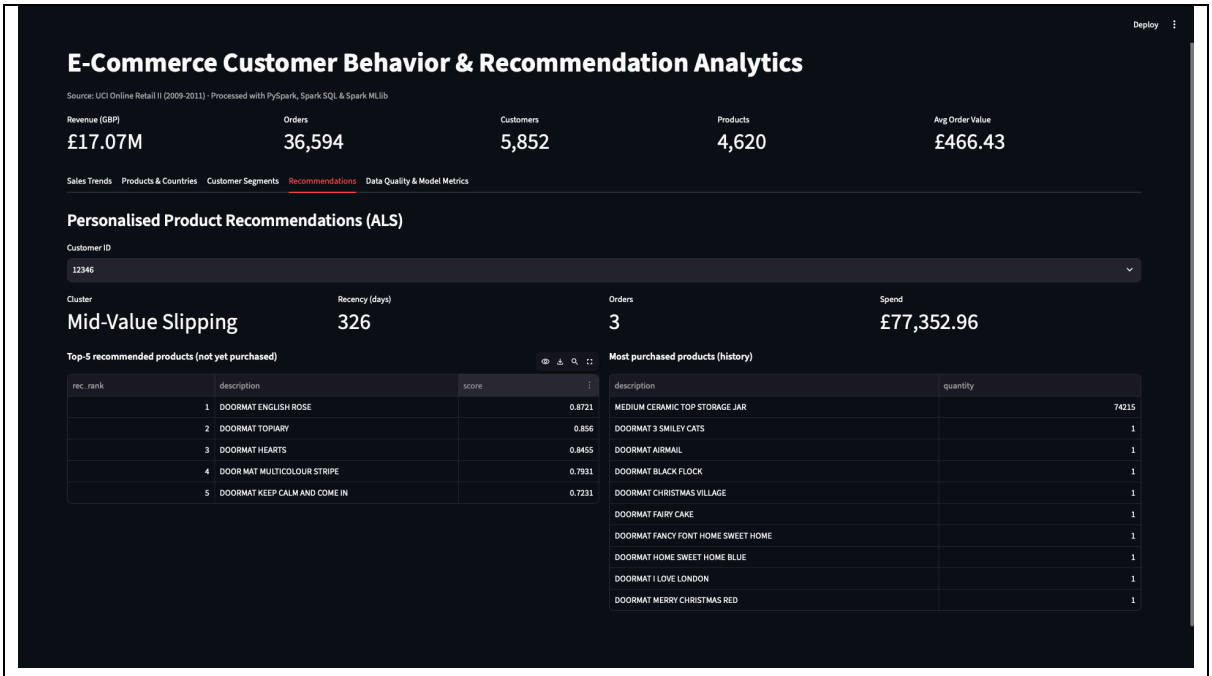


Figure 11.4: Dashboard, Recommendations tab (Purchase history and top-5 ALS recommendations for a selected customer)

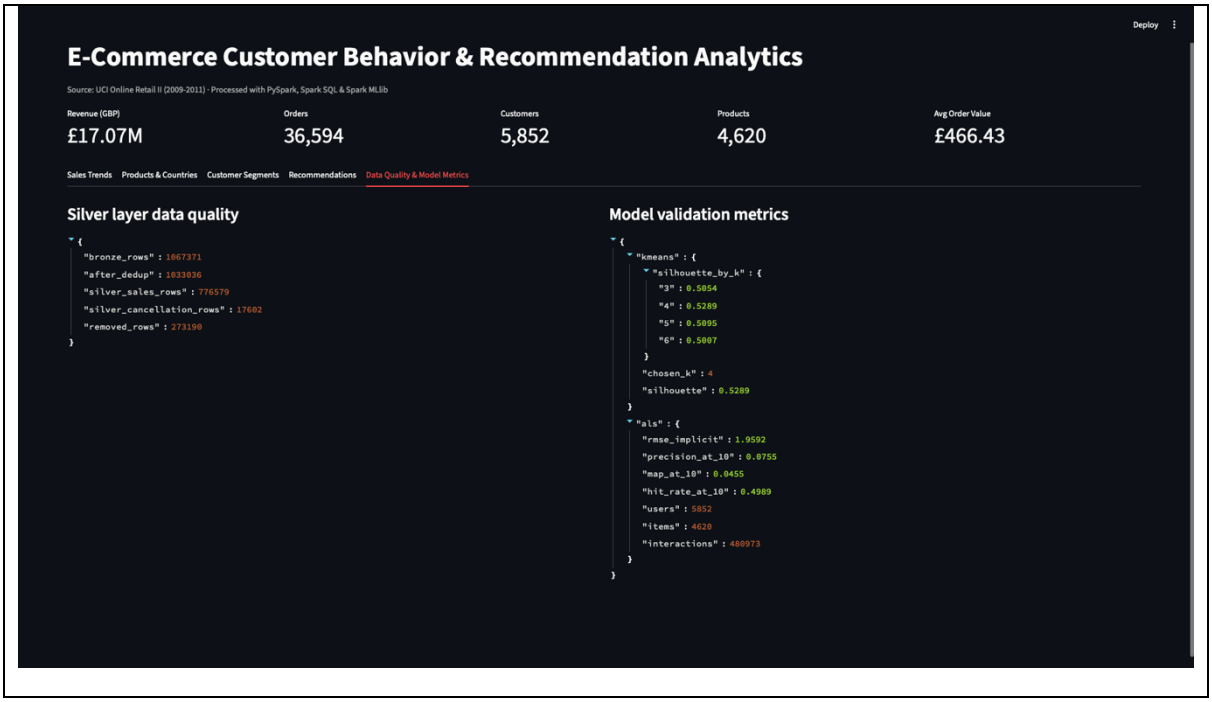


Figure 11.5: Dashboard, Data Quality and Model Metrics tab (Silver data quality report and model validation metrics)

12. Automation and Cloud Deployment

12.1 Pipeline Automation

The complete pipeline (Bronze, Silver, Gold and MLlib) runs with a single command: **run_pipeline.sh** on macOS/Linux or **run_pipeline.ps1** on Windows PowerShell. Each stage stops the run if it fails. A full local run takes about five minutes.

12.2 Apache Airflow

The DAG dags/ecommerce_pipeline_dag.py defines four BashOperator tasks, ingest_bronze, clean_silver, build_gold and train_mllib_models, chained in order and scheduled daily with retries. This allows the pipeline to refresh the dashboard automatically when new data arrives.

12.3 Google Cloud Dataproc

The script cloud/dataproc_submit.sh creates a Cloud Storage bucket, uploads the landed CSV, creates a Dataproc cluster and submits the four PySpark jobs with --data-dir=gs://bucket/data, so that all layers are read from and written to Cloud Storage. Results are copied back for the dashboard and the cluster is deleted afterwards to control cost. The same PySpark code runs locally and on Dataproc without changes.

13. Version Control

The project is maintained on GitHub at **github.com/ronitx05/ecommerce-customer-analytics**. Work was split into feature branches, and each branch was merged into main through a pull request.

PR	Branch	Contents
#1	feature/setup-bronze	Project setup, configuration and Bronze ingestion
#2	feature/silver-layer	Silver cleaning and data quality report
#3	feature/gold-analytics	Gold tables with Spark SQL and window functions
#4	feature/mllib-models	K-Means segmentation and ALS recommendations
#5	feature/dashboard-automation	Streamlit dashboard, automation, Airflow and Dataproc
#7	feature/windows-powershell	PowerShell pipeline script for Windows
#8	feature/notebook	Jupyter notebook walkthrough

Table 13.1: Pull requests

14. Results and Business Insights

- **Seasonality:** revenue peaks in October and November; campaigns and stock planning should start before Q4.
- **Revenue concentration:** the High-Value Loyal segment (about 20% of customers) has an average spend of GBP 10,727, roughly 35 times that of Lapsed customers. These customers should be retained with loyalty rewards and priority service.
- **Mid-Value Slipping (1,430 customers):** they used to buy regularly but have not ordered for about 8 months on average. They are the best target for win-back campaigns.
- **Recent Low-Spend (1,283 customers):** recently active but with small baskets. ALS cross-sell recommendations can be used to increase basket size.
- **Lapsed / Lost (1,974 customers):** one-time buyers from more than a year ago; low-cost re-engagement only.
- **Markets:** the UK dominates, while the Netherlands and EIRE have a few very large accounts; Germany's high cancellation rate needs investigation.
- **Recommendations:** the ALS model finds relevant products for half of customers within its top 10 and can drive personalised emails and on-site suggestions.

15. Conclusion and Future Scope

This project delivered a complete, reproducible Big Data pipeline for e-commerce customer analytics. Using PySpark and the medallion architecture, one million noisy transactions were turned into clean, analysis-ready data and business tables. Spark SQL with window functions produced the KPIs and trends, Spark MLlib produced four

interpretable customer segments and personalised recommendations, and the results are available through an interactive dashboard. The pipeline is automated, version-controlled and ready to run on Google Cloud Dataproc.

Future scope:

- Real-time processing of new orders using Spark Structured Streaming with Kafka.
- Customer churn and customer lifetime value prediction using classification and regression models.
- Hyper-parameter tuning of ALS with CrossValidator, and hybrid recommendations that also use product descriptions.
- Delta Lake tables for ACID updates and time travel, and deployment of the dashboard to the cloud.

16. References

[1] Chen, D. (2012). Online Retail II [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CG6D

[2] Chen, D., Sain, S. L., and Guo, K. (2012). Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. Journal of Database Marketing and Customer Strategy Management, 19(3), 197-208.

[3] Hu, Y., Koren, Y., and Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In Proceedings of the IEEE International Conference on Data Mining (ICDM), 263-272.

[4] Zaharia, M., et al. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56-65.

[5] Apache Spark Documentation: Spark SQL, DataFrames and MLlib. https://spark.apache.org/docs/latest/

[6] Streamlit Documentation. https://docs.streamlit.io

[7] Apache Airflow Documentation. https://airflow.apache.org/docs/

[8] Google Cloud Dataproc Documentation. https://cloud.google.com/dataproc/docs